# Database Assignment

## (Chapter 1)

**[ Name & Last Name: Yasin Rahimi ]**

**[ ISE Department & 4th semester ]**

## Review Questions

**1.1.** Define the following terms: *data*, *database*, *DBMS*, *database system*, *database catalog*, *program-data independence*, *user view*, *DBA*, *end user*, *canned transaction*, *deductive database system*, *persistent object*, *meta-data*, and *transaction-processing application*.

### Answer:

**Database:** A collection of related data. Represents a miniworld, logically coherent, designed for a specific purpose .

**DBMS** (Database Management System): A computerized system that enables users to create and maintain a database. A general-purpose software system .

**Database System:** The database and DBMS software together .

**Database Catalog** (or Dictionary): Stores the database definition or descriptive information (metadata) such as data types, structures, and constraints.

**Program-Data Independence:** The property that allows changes to the structure of data files without requiring changes in the application programs. The structure is stored in the catalog separately from programs.

**User View:** A different perspective or view of the database. A subset of the database or virtual data derived from the database files but not explicitly stored .

**DBA** (Database Administrator): The chief administrator responsible for authorizing access, coordinating and monitoring use, and acquiring resources. Accountable for security breaches and poor response time.

**End User:** People whose jobs require access to the database for querying, updating, and generating reports.

**Canned Transaction:** Standard types of queries and updates carefully programmed and tested for naïve or parametric end users.

**Deductive Database System:** Systems that provide capabilities for defining deduction rules for inferencing new information from stored database facts.

**Persistent Object:** A complex object (e.g., in C++ or Java) stored permanently in an object-oriented DBMS that survives program termination and can be directly retrieved later .

**Meta-Data:** Information in the catalog describing the structure of the primary database (file structures, data types, constraints) .

**Transaction-Processing Application:** Applications (e.g., reservation systems, banking) requiring up-to-date information available to all users, relying on concurrency control and recovery.

**1.2.** What four main types of actions involve databases? Briefly discuss each.

**Answer:**

**Defining:** Specifying data types, structures, and constraints. Stored as metadata in the catalog.

**Constructing:** Storing the data on a storage medium controlled by the DBMS.

**Manipulating:** Querying, updating, and generating reports from the data.

**Sharing:** Allowing multiple users and programs to access the database simultaneously.

**1.3.** Discuss the main characteristics of the database approach and how it differs from traditional file systems.

**Answer:**

**Self-Describing Nature:** DBMS contains metadata in catalog; file systems embed definitions in programs.

Insulation between Programs and Data (Program-Data Independence): DBMS stores structure separately from programs; file systems require program changes when file structure changes.

**Data Abstraction:** DBMS provides conceptual representation hiding storage/implementation details.

**Support of Multiple Views:** DBMS provides facilities for different user perspectives (subsets or virtual data).

**Sharing of Data and Multiuser Transaction Processing:** DBMS includes concurrency control for simultaneous access; file systems lack this

**1.4.** What are the responsibilities of the DBA and the database designers?

**Answer:**

DBA Authorizing access, coordinating/monitoring database use, acquiring software/hardware resources, accountable for security breaches and poor response time.

Database Designers Identifying data to be stored, choosing appropriate structures to represent/store data, communicating with users to understand requirements, integrating views of different user groups.

**1.5.** What are the different types of database end users? Discuss the main activities of each.

**Answer:**

**Casual End Users:** Occasional access, need different information each time. Use sophisticated query interfaces (e.g., middle/high-level managers).

**Naïve or Parametric End Users:** Constantly query/update using canned transactions (e.g., bank tellers, reservation agents, social media users).

**Sophisticated End Users:** Engineers, scientists, analysts. Familiarize themselves with DBMS facilities to implement complex applications.

**Standalone Users:** Maintain personal databases using ready-made packages with menu/graphic Interfaces (e.g., financial software user)

**1.6.** Discuss the capabilities that should be provided by a DBMS.

**Answer:**

- Controlling Redundancy
- Restricting Unauthorized Access
- Providing Persistent Storage for Program Objects
- Providing Storage Structures and Search Techniques for Efficient Query Processing
- Providing Backup and Recovery
- Providing Multiple User Interfaces
- Representing Complex Relationships among Data
- Enforcing Integrity Constraints
- Permitting Inferencing and Actions Using Rules and Triggers

**1.7.** Discuss the differences between database systems and information retrieval systems.

## Exercises

**1.8.** Identify some informal queries and update operations that you would expect to apply to the database shown in Figure

**Answer:**

Retrieve the transcript (list of courses and grades) of 'Smith ' List the names of students who took the section of the 'Database' course offered in fall 2008 and their grades List the prerequisites of the 'Database' course.

**Updates:**

- Change the class of 'Smith' to sophomore.
- Create a new section for the 'Database' course for this semester.
- Enter a grade of 'A' for 'Smith' in the 'Database' section of last semester.

**1.9.** What is the difference between controlled and uncontrolled redundancy? Illustrate with examples.

**Answer:**

Uncontrolled Redundancy: Storing same data multiple times without control. Causes wasted space, duplication of effort, inconsistency. Example: Accounting and registration

offices each keep separate student files; one enters birth date as 'JAN-19-1988', another as 'JAN-29-1988.'

Controlled Redundancy: Redundancy used for performance (denormalization) but DBMS controls it by automatically checking consistency. Example: Storing Student_name and Course_number redundantly in GRADE_REPORT (Figure 1.6a), and DBMS checks that Student_name-Student_number matches a STUDENT record

**1.10.** Specify all the relationships among the records of the database shown in Figure 1.2.

**Answer:**

- STUDENT related to one or more GRADE_REPORT records (courses completed).
- COURSE related to one or more SECTION records (course offerings).
- COURSE related to one or more PREREQUISITE records (as main course).
- PREREQUISITE relates two COURSE records: one as Course_number, one as Prerequisite_number.
- SECTION related to one COURSE and one or more GRADE_REPORT records.
- GRADE_REPORT related to one STUDENT and one SECTION

**1.10.** Give some additional views that may be needed by other user groups for the database shown in Figure 1.2.

**Answer:**

- Department Chair View: Instructors per department with courses they teach.
- Registrar View: Student list with completed credit hours and calculated GPA.
- Advisor View: Student personal information (Name, Class, Major) without grades

**1.12.** Cite some examples of integrity constraints that you think can apply to the database shown in Figure 1.2.

**Answer:**

- Key/Uniqueness Constraint: Student_number in STUDENT must be unique. Course_number in COURSE must be unique.
- Referential Integrity Constraint: Section_identifier in GRADE_REPORT must match a SECTION record. Course_number in SECTION must exist in COURSE.
- Domain/Data Type Constraint: Grade must be from {A', B', C', D', F'}. Class must be integer between 1 and 5.
- Relationship Constraint: Every SECTION record must relate to a COURSE record

**1.13.** Give examples of systems in which it may make sense to use traditional file processing instead of a database approach.

**Answer:**

- Simple, well-defined applications not expected to change.
- Real-time systems with stringent requirements where DBMS overhead is unacceptable.
- Embedded systems with limited storage capacity.
- CAD tools with proprietary file management software.
- Communication/switching systems requiring very fast hierarchical data access.

**1.14.** Consider Figure 1.2.

a. If the name of the 'CS' (Computer Science) Department changes to 'CSSE'

(Computer Science and Software Engineering) Department and the corresponding prefix for the course number also changes, identify the columns in the database that would need to be updated

**Answer a:**

- COURSE table: Course_number (e.g., CS1310 → CSSE1310) and Department column (CS → CSSE).
- SECTION table: Course_number column.
- PREREQUISITE table: Course_number and Prerequisite_number columns.

- GRADE_REPORT table: If it contains Course_number (as in Figure 1.6a example, though not in Figure 1.2), that column also needs updating.

b. Can you restructure the columns in the COURSE, SECTION, and

PREREQUISITE tables so that only one column will need to be updated?

**Answer b:**

Yes. Split Course_number into two columns:

- Department_prefix (text, e.g., CS)
- Number (numeric, e.g., 1310)
- Then:
- COURSE: Primary key = (Department_prefix, Number). Remove Department column (redundant).
- SECTION: Replace Course_number with two foreign key columns: Dept_prefix and Course_num.
- PREREQUISITE: Replace Course_number with Dept_prefix and Course_num; replace Prerequisite_number with Prereq_dept_prefix and Prereq_num.
- Now, changing CS to CSSE requires updating only the Department_prefix column In the COURSE table once. All related tables automatically reflect the change via foreign key relationships.

**Respectfully**